

Encoder–Decoder Multi-Head Attention (Cross-Attention)

Goal: Understand exactly how the decoder uses the encoder output through Multi-Head Attention, including Q/K/V, dimensions, heads, attention scores, concatenation, and the final output.

1. Where Encoder–Decoder Attention Fits

A Transformer encoder first reads the complete source sentence and produces contextual representations. The decoder generates the target sentence step by step. After the decoder's **masked self-attention**, it performs **encoder–decoder attention**, also called **cross-attention**.

Flow:

Encoder: Input → Self-Attention → Add & Norm → Feed Forward → Add & Norm → **Encoder Output E**

Decoder: Target → Masked Self-Attention → Add & Norm → **Cross-Attention** → Add & Norm → Feed Forward → Add & Norm → Linear → Softmax

Key idea: In cross-attention, the decoder asks a question using its current representation, while the encoder provides the information from the source sentence.

2. The Most Important Rule: Where Q, K and V Come From

Unlike encoder self-attention, where Q, K and V all come from the same sequence, cross-attention uses two different sources.

Matrix	Comes from	Meaning
Q (Query)	Decoder	What information does the decoder need right now?
K (Key)	Encoder	What source positions are available to match against?
V (Value)	Encoder	What actual information is retrieved from those positions?

Therefore:

$$Q = D \times W_Q$$

$$K = E \times W_K$$

$$V = E \times W_V$$

where D is the decoder representation and E is the encoder output.

3. Example: Translating “Je suis étudiant” → “I am a student”

Suppose the encoder receives the French sentence:

Je | suis | étudiant

The encoder's final output contains one contextual vector for each source token:

$$E = [e_{\text{Je}}, e_{\text{suis}}, e_{\text{étudiant}}]$$

Now imagine the decoder has already generated:

<START> I am

At this point it is trying to predict the next word, which should be “a”. The decoder's current representation is D.

Cross-attention lets this decoder representation look back at the encoder representations of **Je, suis, étudiant**.

4. Step-by-Step Cross-Attention

Step 1 — Create Q from the decoder

The decoder representation is projected using W_Q :

$$Q = D \times W_Q$$

This Q represents the decoder's current information need. Because the decoder is currently generating the next target word, its query asks, in effect: *"Which parts of the source sentence are relevant for what I should generate next?"*

Step 2 — Create K and V from the encoder

The complete encoder output is projected twice:

$$K = E \times W_K$$

$$V = E \times W_V$$

K is used for matching/relevance. V contains the information that will actually be combined to produce the attention output.

Step 3 — Calculate QK^T

The query is compared with every encoder key:

$$\text{Scores} = QK^T / \sqrt{d_K}$$

If there are 3 source tokens, one decoder query produces 3 scores — one score for *Je*, one for *suis*, and one for *étudiant*.

Step 4 — Softmax converts scores into attention weights

For illustration, suppose the scores after scaling are:

[1.0, 0.5, 2.5]

Softmax might produce approximately:

[0.17, 0.10, 0.73]

The exact numbers depend on the learned weights. The important point is that the weights sum to 1. Here the decoder is paying most attention to *étudiant*.

Step 5 — Multiply attention weights by V

The output is the weighted combination of the encoder values:

$$\text{Output} = [0.17, 0.10, 0.73] \times V$$

Conceptually:

$$\text{Output} = 0.17 \cdot V(\text{Je}) + 0.10 \cdot V(\text{suis}) + 0.73 \cdot V(\text{étudiant})$$

So the decoder receives a source-aware representation dominated by the information associated with *étudiant*.

5. Why This Is Called “Cross” Attention

The attention crosses the encoder–decoder boundary: the **decoder supplies Q** , while the **encoder supplies K and V** .

Think of it like this:

Decoder: “What do I need to know?” $\rightarrow Q$

Encoder: “Here are the source locations you can match against.” $\rightarrow K$

Encoder: “Here is the information stored at those locations.” $\rightarrow V$

6. Multi-Head Attention

Instead of performing one attention operation, the Transformer creates multiple heads. Each head learns a different projection of Q , K and V .

For head i :

$$Q_i = D \times WQ_i$$

$$K_i = E \times WK_i$$

$$V_i = E \times WV_i$$

$$\text{head}_i = \text{softmax}(Q_i K_i^T / \sqrt{d_k}) \times V_i$$

Finally, all heads are concatenated and projected:

$$\text{MHA} = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \times W_O$$

7. Small Dimension Example: 4 Heads

Suppose the model dimension is $d_{\text{model}} = 8$ and there are **4 heads**. Then a common design is:

Item	Dimension
Decoder representation D	1×8
Encoder output E (3 source tokens)	3×8
Number of heads	4
Dimension per head $d_k = d_v$	$8 / 4 = 2$
Q for one head	1×2
K for one head	3×2
V for one head	3×2
QK^T for one head	1×3
Attention weights $\times V$	1×2
Concatenate 4 heads	1×8
After W_O	1×8

Important: The heads are not comparing one token to another one-to-one. Each head still compares the decoder query against **all encoder positions**. The split into smaller dimensions happens because the learned projection matrices WQ , WK and WV map the model dimension into each head's smaller subspace.

8. Why Do We Need WQ , WK and WV ?

Suppose $d_{\text{model}} = 8$ and we use 4 heads. Each head needs 2-dimensional Q , K and V . Therefore, the learned matrices perform the dimension conversion.

For one head:

$$(1 \times 8) \times (8 \times 2) = 1 \times 2$$

For the encoder keys:

$$(3 \times 8) \times (8 \times 2) = 3 \times 2$$

Thus the fourth dimension does not disappear. It is **projected** from 8 dimensions into 2 dimensions for that head.

9. Why Do We Concatenate the Heads?

Each head captures information in its own learned representation space. With 4 heads and 2 values per head:

$$\text{head}_1: 1 \times 2 \mid \text{head}_2: 1 \times 2 \mid \text{head}_3: 1 \times 2 \mid \text{head}_4: 1 \times 2$$

Concatenation gives:

$$\text{Concat} = 1 \times (2 + 2 + 2 + 2) = 1 \times 8$$

This restores the model dimension. Then W_O mixes information across the heads:

$$(1 \times 8) \times (8 \times 8) = 1 \times 8$$

10. Cross-Attention vs Encoder Self-Attention

Property	Encoder Self-Attention	Encoder-Decoder Cross-Attention
Q comes from	Encoder	Decoder
K comes from	Encoder	Encoder
V comes from	Encoder	Encoder
Can attend to	Source tokens	Source/encoder tokens
Purpose	Build contextual source representations	Bring relevant source information into decoder

11. What Happens After Cross-Attention?

The cross-attention output is added to the decoder's residual stream and passed through Layer Normalization. Then the Feed-Forward Network (FFN) is applied.

Conceptually:

Cross-Attention → Add & Norm → Feed Forward → Add & Norm → Linear → Softmax

The FFN is applied independently to each position. Attention mixes information across positions; the FFN then performs a learned nonlinear transformation on each resulting representation.

12. One Complete Mental Picture

Encoder:

"Here is everything I understood from the source sentence." → Encoder output E

Decoder:

"Based on what I have generated so far, what source information do I need?" → Query Q

Cross-Attention:

Compare Q with every encoder Key K → softmax → attention weights → weighted sum of encoder Values V.

Result:

A decoder representation enriched with the most relevant information from the source sentence.

13. Exam / Interview Summary

1. Encoder produces contextual representations E for the source sequence.
2. Decoder's masked self-attention produces its current representation D.
3. In cross-attention, Q comes from D; K and V come from E.
4. Compute scaled dot-product attention: $\text{softmax}(QK^T/\sqrt{d_k})V$.
5. In MHA, this happens independently in multiple heads.
6. Each head uses projection matrices to reduce d_{model} into the head dimension.
7. All head outputs are concatenated back to d_{model} .
8. WO mixes information across heads and produces the final MHA output.
9. The output goes through residual connection + LayerNorm, then FFN.

Memory shortcut: "Decoder asks, Encoder answers."

Decoder → Q | Encoder → K, V